

A 2D Lunar Hopper

Cascaded PD Control, Kalman-Filter Navigation and Monte Carlo Analysis

Tamilarasan Ketheeswaran | self-initiated study | 2026

Code and figures: <https://github.com/Krautsalat13/LunarHopper>

This report studies closed-loop control and navigation for a planar lunar hopper landing. It has two aims: to measure how reliably a classical controller lands the vehicle across a range of uncertain initial conditions, and to quantify how much that reliability drops when the controller runs on a state estimated from noisy sensors instead of the true state. The model is kept deliberately simple (flat moon, 2D, a single gimbaled thruster) so the results isolate the closed-loop behaviour and its sensitivity to the initial conditions.

Model

The state is $\vec{q} = (x, z, \dot{x}, \dot{z}, \vartheta, \omega)$: lateral position, altitude, their rates, body tilt ϑ from the vertical, and tilt rate ω . A single thrust T acts at the base through a gimbal angle δ . Treating the hopper as a rigid body gives

$$m\ddot{x} = T \sin(\vartheta + \delta), \quad m\ddot{z} = T \cos(\vartheta + \delta) - mg, \quad I\ddot{\vartheta} = \ell T \sin \delta, \quad (1)$$

with gimbal-to-centre-of-mass distance $\ell = h/2$ and moment of inertia $I = \frac{1}{12}m(3r^2 + h^2)$.

Parameters: $m = 150$ kg, $g = 1.625$ m/s², $h = 2$ m, $r = 0.25$ m. The inputs are $T \geq 0$ and δ ; the equations are integrated with RK4 ($\Delta t = 0.01$ s) from $z_0 = 10$ m, with the mass held constant. The propagator was checked against two analytic limits first: free fall reaches the surface at $t = \sqrt{2z_0/g} \approx 3.51$ s, and hover thrust $T = mg$ holds altitude.

Choosing the controller

The vehicle is underactuated: two inputs, three degrees of freedom. Neither the thrust T nor the gimbal δ pushes the vehicle sideways on its own, so the only way to move sideways is to tilt the body and use the sideways part of the thrust. So I use three nested loops (Figure 1), each with one input: an *altitude* loop that sets the thrust, an outer *lateral* loop that turns the sideways position error into a commanded tilt $\vartheta_{\text{cmd}} = -K_{p,x}x - K_{d,x}\dot{x}$, and a fast inner *attitude* loop that makes the body follow that tilt with the gimbal. The inner loop runs five times faster than the outer one ($\omega_{n,\vartheta} = 2.5$ vs. $\omega_{n,x} = 0.5$ rad/s), so I can tune them separately.

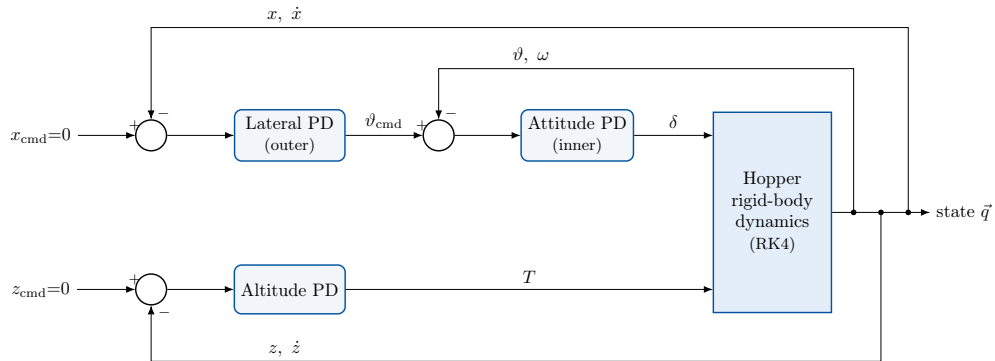


Figure 1: The cascaded controller. The outer lateral loop commands a tilt ϑ_{cmd} that the fast inner attitude loop follows through δ ; the altitude loop sets T on its own. Each summing junction subtracts the fed-back state from the reference.

Each loop is a second-order system. Close to vertical and near hover, each channel is just a double integrator driven by its input. In the altitude loop, if I cancel gravity with a feed-forward term, $T = mg + u_z$, then PD feedback $u_z = -K_p z - K_d \dot{z}$ gives

$$\ddot{z} + 2\zeta\omega_n \dot{z} + \omega_n^2 z = 0, \quad \omega_n^2 = \frac{K_p}{m}, \quad 2\zeta\omega_n = \frac{K_d}{m}, \quad (2)$$

The lateral and attitude loops look the same, except the m in front is replaced by m/T and $I/(\ell T)$. Since those depend on the current thrust, I schedule the lateral and attitude gains on T to keep ω_n and ζ constant; the altitude loop just uses fixed gains.

Why PD. With $K_d = 0$ each loop is just $\ddot{z} + \omega_n^2 z = 0$, an undamped oscillator, so the lander would keep oscillating and never settle. The derivative term $2\zeta\omega_n \dot{z}$ damps that out, so PD is the simplest thing that works. I do not add an integral term because there is no steady disturbance or bias to cancel in this model; that would change with something like crosswinds or a mass/gravity mismatch. I pick the damping per loop. The altitude loop is critically damped ($\zeta = 1$), since even one overshoot could put it into the ground on the way down. The lateral and attitude loops use $\zeta = 0.7$, where a small overshoot is fine and the faster response matters, because the offset has to be gone before touchdown.

Loop	States	Output	ω_n (rad/s)	ζ	Limits
Altitude	$z, \dot{z}$	thrust T	0.5	1.0	$T \geq 0$
Lateral (outer)	$x, \dot{x}$	tilt cmd ϑ_{cmd}	0.5	0.7	$\pm 20^\circ$
Attitude (inner)	ϑ, ω	gimbal δ	2.5	0.7	$\pm 10^\circ$

Table 1: The three PD loops. The inner loop is five times faster than the outer, so they can be tuned independently.

Saturation and cutoff. The commanded tilt is limited to $\pm 20^\circ$ and the gimbal to $\pm 10^\circ$. The engine is cut below 0.10 m, so in the ideal case touchdown is a short free fall at $\sqrt{2g \cdot 0.1 \text{ m}} \approx 0.57 \text{ m/s}$.

Two representative descents

Working case (Figure 2; $x_0 = 2 \text{ m}$, $\vartheta_0 = 5^\circ$). The thrust stays at zero at first: the lander starts above the target altitude, so the altitude loop would want downward thrust, which clips to zero, and gravity does the early descent for free. The attitude stays constant while there is no thrust, since you cannot tilt without it. Once the thrust comes up, the gimbal tilts the body to cancel the sideways offset and only straightens it out once the offset is gone, so the lander ends up upright last; the sideways position overshoots a bit in the middle, as expected. Just before 13.5 s it crosses the 0.10 m cutoff and the engine shuts off. There is no overshoot in z .

Failing case (Figure 3; $x_0 = 20 \text{ m}$, twice the initial height). The commanded tilt saturates at -20° to try to recover the large offset. The controller does bring x back toward zero, but the recovery takes longer than the descent, so the lander is still leaning at touchdown ($\vartheta = +20^\circ$) with $v_x = -2.5 \text{ m/s}$: a crash. What matters here is how big an offset can be fixed in the time left before landing, not how big the initial tilt is.

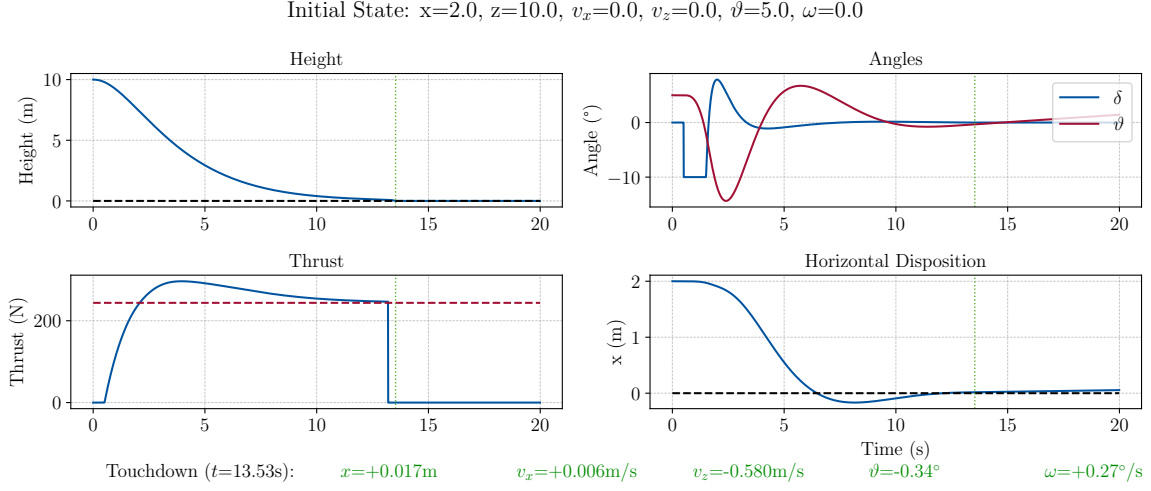


Figure 2: Working case ($x_0 = 2$ m, $\vartheta_0 = 5^\circ$). Touchdown at $t = 13.5$ s: $x = +0.02$ m, $v_x = +0.01$ m/s, $v_z = -0.58$ m/s, $\vartheta = -0.3^\circ$, a soft landing. The dotted green line marks touchdown; plots are truncated there.

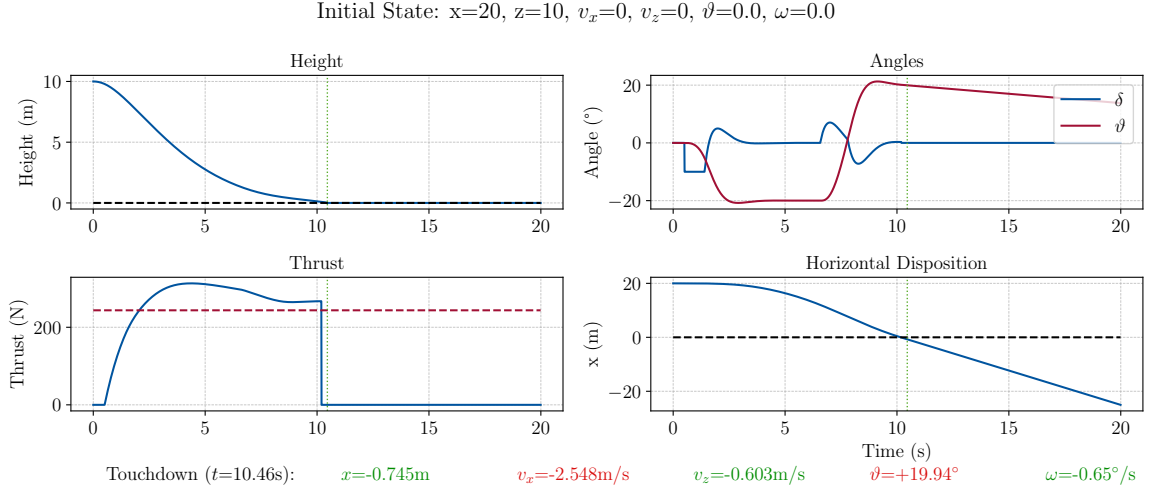


Figure 3: Failing case ($x_0 = 20$ m). The commanded tilt saturates at -20° to recover the offset; touchdown at $t = 10.5$ s with $v_x = -2.5$ m/s and $\vartheta = +20^\circ$, a crash. The lateral-recovery manoeuvre outlasts the descent.

Monte Carlo dispersion analysis

To test the controller more broadly, I run $N = 1000$ descents with the initial conditions picked uniformly from $x_0 \in [-5, 5]$ m, $\dot{x}_0 \in [-2, 2]$ m/s, $\dot{z}_0 \in [-3, 0]$ m/s, $\vartheta_0 \in [-10^\circ, 10^\circ]$, $\omega_0 \in [-3^\circ, 3^\circ]$ /s, all from $z_0 = 10$ m. I call a landing *soft* if $|x| < 0.5$ m, $|v_x| < 0.5$ m/s, $|v_z| < 1$ m/s, $|\vartheta| < 5^\circ$, and *acceptable* if it meets the looser limits $|x| < 1$ m, $|v_x| < 1$ m/s, $|v_z| < 2$ m/s, $|\vartheta| < 10^\circ$ but is not soft.

Flying on the true state, **830 (83.0%) were soft** and 52 acceptable, so **882 (88.2%) within tolerance** and **118 (11.8%) failures**, all of them out-of-tolerance touchdowns, no timeouts. The touchdown scatter (Figure 4, right) shows the sideways velocities staying inside $|v_x| < 1$ m/s, while the failures all have touchdown tilt above 10° . That tilt does not come from large initial tilts (those correct quickly), it comes from large initial sideways offsets: to cancel an offset the lander has to hold a tilt, and that tilt is not always straightened out in time.

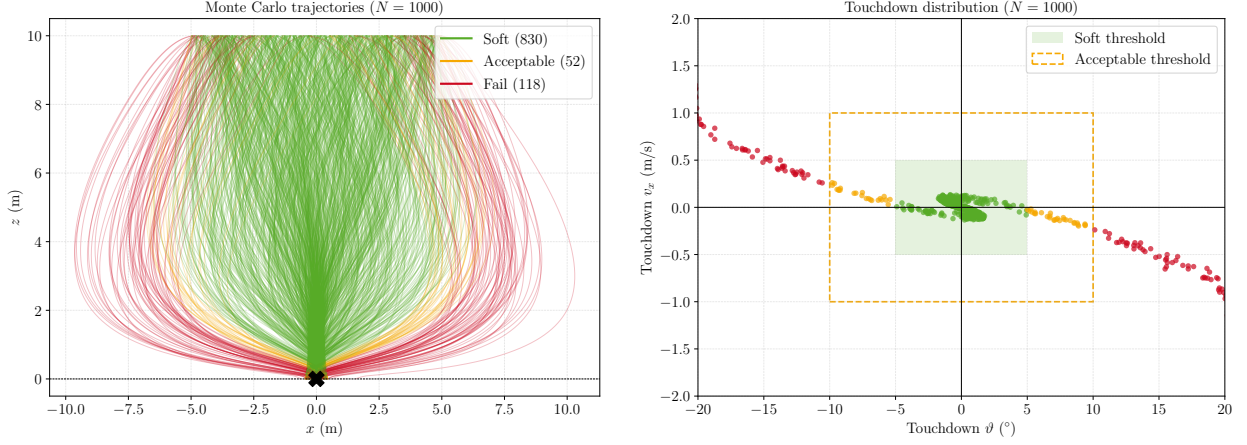


Figure 4: Monte Carlo, perfect state ($N = 1000$). **Left:** trajectories coloured by outcome. **Right:** touchdown distribution in the ϑ - v_x plane against the soft (shaded) and acceptable (dashed) limits; marker colour reflects the classification.

Navigation: flying on an estimate

So far the controller gets the true state, which a real vehicle does not have. The navigation layer adds sensors and an Extended Kalman Filter that estimates the four translational states $s = (x, z, \dot{x}, \dot{z})$; attitude is still taken as known. I model two sensors: an accelerometer read every step, which measures the specific force (the thrust acceleration, not gravity) plus white noise, and an altimeter read once every 0.2 s, which measures altitude plus white noise.

The filter keeps a Gaussian estimate of s (a mean and a covariance) and updates it in two steps. The *predict* step dead-reckons on the accelerometer: it moves the mean forward with the measured acceleration and grows the covariance to account for the noisy reading and the imperfect model. The *update* step runs only when an altimeter reading comes in: it compares the altitude the filter expected with what the altimeter actually read, and moves the mean toward the reading, with how much depending on which one it trusts more, while shrinking the covariance. Two things matter here. With attitude known, both the dynamics and the measurement are linear in s , so the EKF is really just a linear Kalman filter. And the altimeter only sees altitude, so horizontal position is never measured directly: it is dead-reckoned and drifts with any error in the initial velocity, which ends up being the main source of error.

Results. Running the same 1000 runs on the estimate instead of the truth (Table 2, Figure 5), the **soft rate drops from 83.0% to 58.6%**, while the **within-tolerance rate barely changes, 88.2% to 84.0%**. The reason is what the sensors can see. The altimeter holds the vertical channel, so the descent and the cutoff timing stay about the same and few landings drop out of tolerance. Horizontal position is not measured, so it drifts, and that drift pushes tight soft landings into the wider acceptable band. The estimate-versus-truth plots show this: z , $\dot{x}$ and $\dot{z}$ track well, and only x keeps a small bias that no sensor can remove. To check the filter is consistent, I run it over 300 random descents: the 4-state NEES sits at 4.0 and the altimeter NIS at 1.0, both inside their 95% bands, so the covariance it reports matches its real error. The lost soft landings are not a tuning issue, they are the price of never measuring horizontal position.

Outcome	Perfect state	EKF estimate
Soft	83.0%	58.6%
Within tolerance (soft or acceptable)	88.2%	84.0%
Failure	11.8%	16.0%

Table 2: Landing outcomes over the same $N = 1000$ runs, controller flown on truth versus on the EKF estimate.

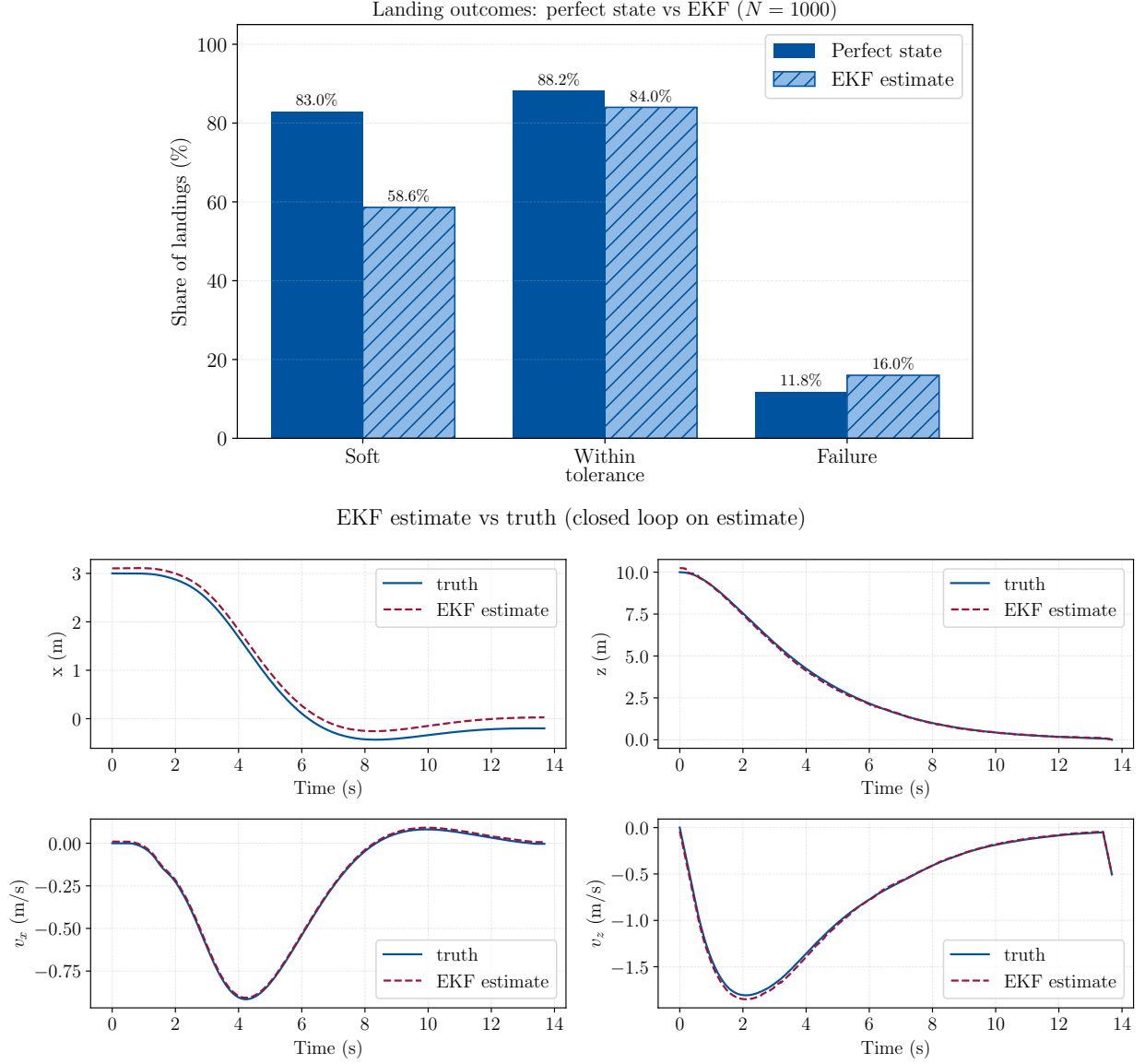


Figure 5: Top: outcome shares, perfect state (solid) versus EKF estimate (hatched); the loss is soft landings shifting into the acceptable band. **Bottom:** estimate versus truth on one descent. Only x carries a lasting bias, since horizontal position is never measured.

Limitations and outlook

The lander is reactive: it corrects errors as they appear, with no trajectory planning or fuel optimisation, and it relies on the offsets being correctable before touchdown. Its limits point straight at the methods it does not use. The next step would be optimisation-based guidance, where fuel, actuator and state limits enter as explicit constraints, so the large-offset regime is handled by planning the trajectory instead of reacting to it, and the touchdown limits are met by construction rather than checked afterwards. On the navigation side, the one real gap is horizontal position: a terrain-relative or Doppler velocity sensor would measure the channel the altimeter cannot. Going from reactive robustness towards designed-in robustness is the direction I would like to take this.

Implemented in Python (NumPy): RK4 propagator, cascaded gain-scheduled PD controller, Extended/linear Kalman Filter, Monte Carlo campaign, all seeded and reproducible. Full code at <https://github.com/Krautsalati3/LunarHopper>.